

Welcome to Code Crunch!

C11 | [CRUNCH ARCADE]

[Tuesdays 3-4 PM]/[Wednesdays
10-11 AM] Weekly | Spring 2025

Attendance

Earn a Certificate of
Completion by attending
at least 70% of the
workshops



CODECRUNCH.CS.FIU





Unit #2

Understanding Unity Components and Scripting Basics in Unity

C11 | CRUNCH ARCADE



Agenda for Week 2

Crunch Arcade Week 2 consists of going over...

- **Unity Interface and Components.**
- **Properties of Transform, Rigidbody, and Colliders.**
- **Introduction** to Lighting and Materials.
- **Physics** in Unity: **Gravity** and **Collision-detection.**



Recap of Week 1

Week 1 Covered:

- **Introduction** to Unity, what it is, how it is used, and its place in the industry.
- **How to Install** Unity and its necessary components.
- What **GameObjects** are-- which are essentially containers for everything and anything in a game in **Unity**.

Navigating the Unity Interface



Unity's Main Sections:

Hierarchy

Project

Inspector

**Scene &
Game View**

- In Unity, there are four main sections you need to be familiar with to effectively build and manage your game.
- These sections help you organize your project, modify objects, and test your game in real time.

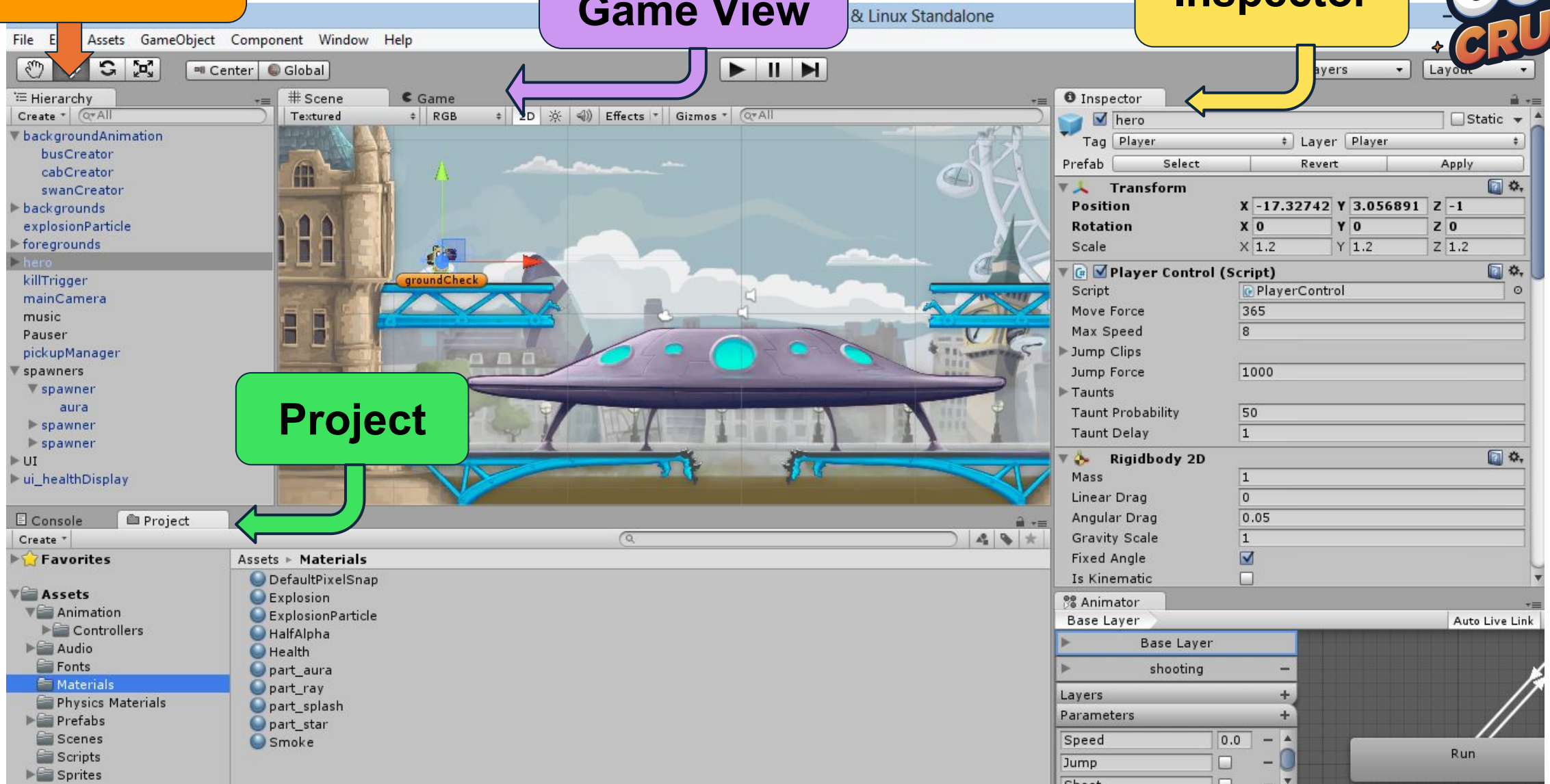
Hierarchy

Scene &
Game View

Inspector

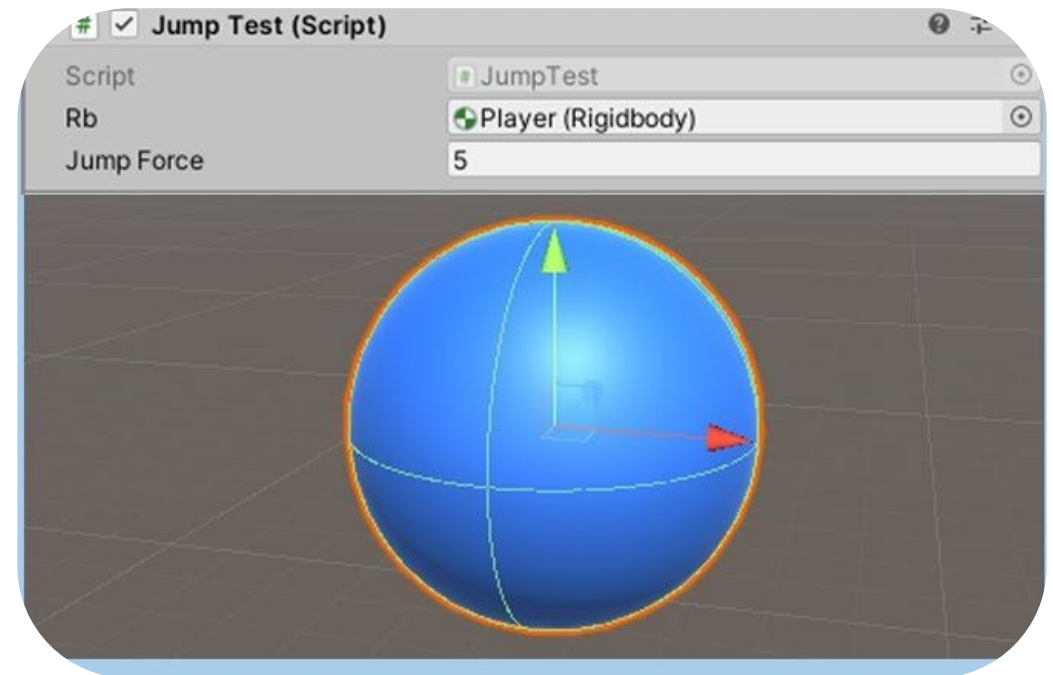
CODE
CRUNCH

Project



What are Components?

- Unity uses **components** to define the behavior and properties of **GameObjects**.
- Components add **functionality** to GameObjects.
- Each GameObject can have **multiple components** that define its **behavior**.
- Components work together to create movement, physics, sound, and visuals.



Why are they important?

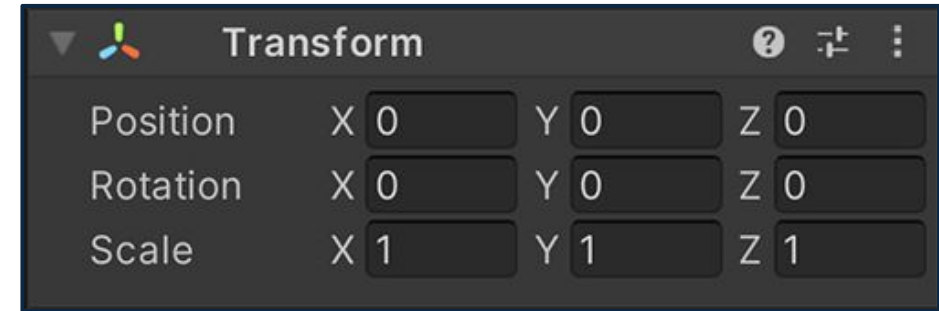
- They make Unity **modular** and **customizable**. Add only what you need!
- Unity's built-in systems (Physics, Rendering, AI, etc.) all rely on **components**.
- Even **custom scripts** (your own code) are components that let you program game behavior.



Properties of Transform, Rigidbody, and Colliders.

What is Transform?

- The **Transform** property controls the GameObject's **Position**, **Rotation**, and **Scale**.

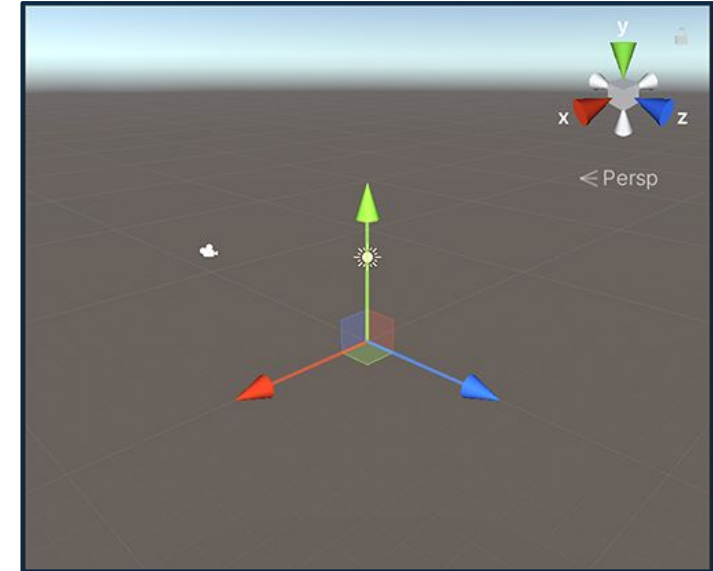


* This menu can be located on the right-side of your screen under 'Inspector'.

- Position:** GameObject's position in the x, y, and z coordinates.
- Rotation:** GameObject's rotation around the x-axis, y-axis, and z-axis-- in degrees.
- Scale:** GameObject's scale/size along the x-axis, y-axis, and z-axis-- '1' is the original size.

Editing Transforms

- If using **Unity 2D's space**, you can only manipulate the x-axis or the y-axis.
- In **Unity's 3D's space**, you can manipulate all three axis. They are represented with red for x, green for y, and blue for z.



What is **Rigidbody**?

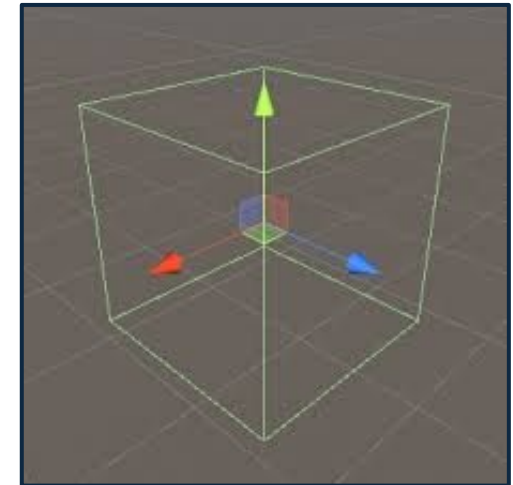
- The **Rigidbody** component enables physics-based movement, allowing objects to fall, collide, and respond to forces.
- It essentially overrides the **Transform** component based off of **gravity**, **collisions**, and **forces (and other variables)**.
- For example, **Rigidbody** allows a ball object to fall naturally.



* This menu can be located on the right-side of your screen under 'Inspector'.

What is a Collider?

- The **Collider** component defines an object's physical boundaries for collision detection with any other objects. It is invisible in-game.
 - It works with the **RigidBody** component to interact with physic-based processes.
- Common **Types of Colliders** include shapes like:
 - Circle Collider (**2D**) – Used for round objects in 2D games.
 - Sphere Collider (**3D**) – Detects collisions for spherical objects.
 - Box Collider (**2D/3D**) – Works well for rectangular and cubic objects.
 - Capsule Collider (**2D/3D**) – Commonly used for player characters and rounded objects.
- Example use of a **Collider**:
 - Creating a coin object, assigning a **collider**, and signifying a player has collected it when the two objects intersect.

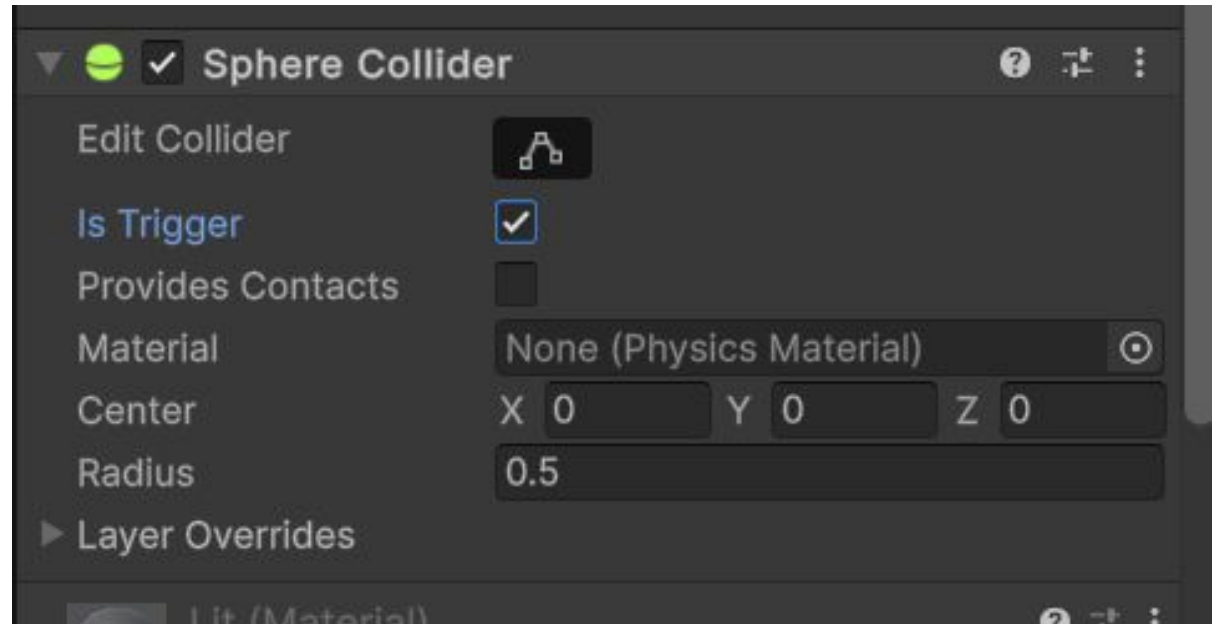


Physics in Unity: Gravity and Collision



To create physics in Unity we use both rigidbody and colliders to make that happen.

First thing we need to do after creating our objects would be to check if they both have colliders in them and make sure **Trigger** is set to true.

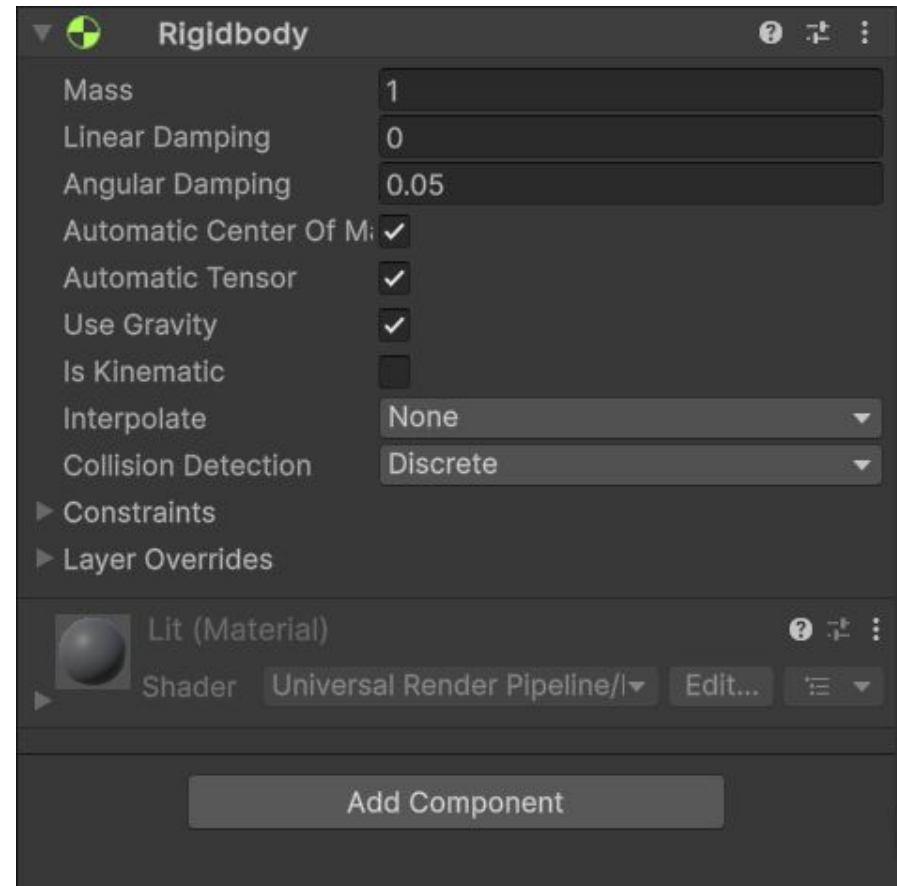


Physics in Unity: Gravity and Collision



Next thing you need to do if make sure the object you are colliding with has a rigidbody. If it doesn't, you can add it by clicking **add component**.

When an object has a rigidbody it means it is affected by the physics engine. If you played the game, you would see the object fall through the ground because now it is affected by gravity. Now your object has gravity.

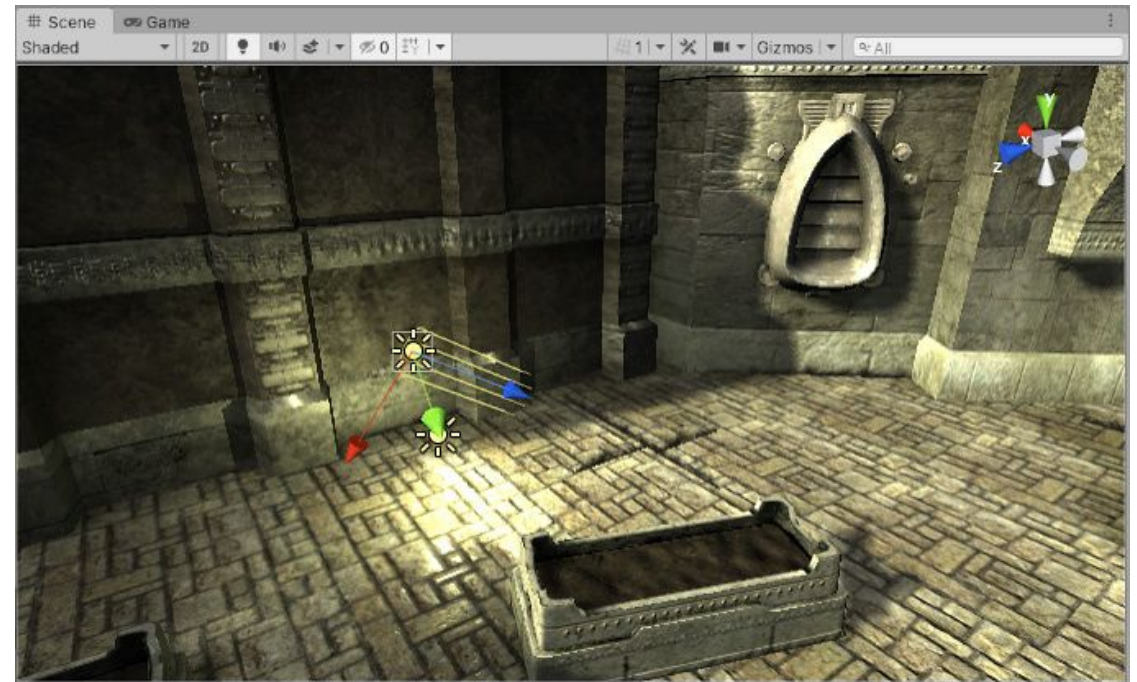




Intro to **Lighting** and **Materials**

Intro To Lighting and Materials

- **Lighting** is an **essential** part in creating a scene.
- Lighting works by approximating how light behaves in the real world
- It can be used for a more realistic result used in a more stylized way.
- Both **2D and 3D games** use lighting to add detail to their settings.



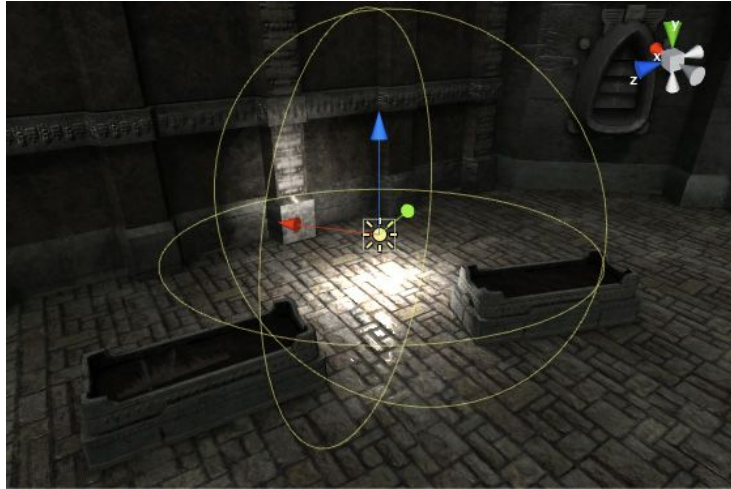
Lighting modes

- **Direct and indirect lighting:** Light that is emitted and hits a surface once and is reflected to the camera is a Direct light. Sky light and lighting that bounces around multiple surfaces is indirect.
- **Real-time and baked:** Real-time is calculating lights during runtime while baked is calculated then applied to the scene.
- **Global Illumination:** Uses both direct and indirect to provide realistic lighting.

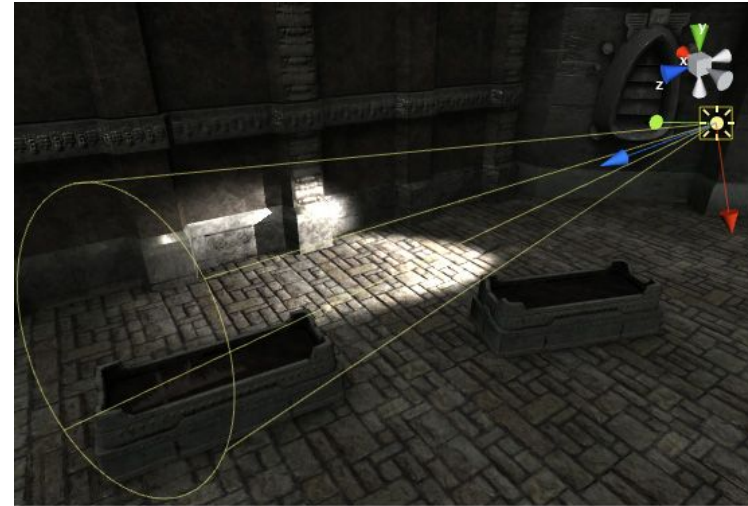
Types of Lighting components

- **Directional Light:** It can be placed anywhere in the scene. Useful for sun like effects
- **Point Light:** A light that sends light out in all directions. Perfect to use on lamps and explosion lighting.
- **Spot Light:** Similar to point light but send light out in a cone angle
- **Area Light:** Emits for a rectangular area or disc. It lights in different directions useful for a street light or house lighting

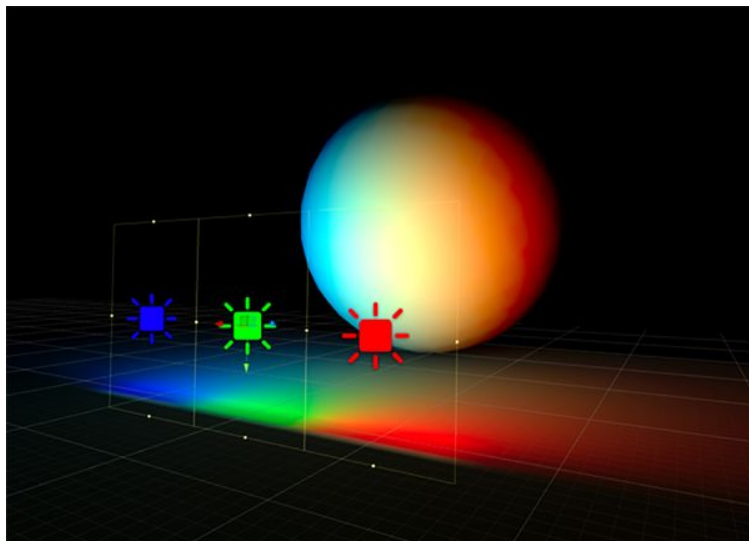
Examples of each Component



Point



Spot



Area



Directional

Lighting Pipeline

Spotlight on Lighting pipeline

How to light a project?

Render Pipelines

Pick a Render Pipeline

- Built-in Render Pipeline
 - Universal Render Pipeline
 - High-Definition Render Pipeline
 - Custom SRP
- Scriptable Render Pipelines (SRPs)

Lighting Settings

Pick a Global Illumination mode

- Baked GI
- Realtime GI
- Baked & Realtime GI
- None

Pick a Lighting Mode

- Baked Indirect
- Subtractive
- Shadowmask
- Distance Shadowmask

Lighting

Add Lights

- Baked
- Realtime
- Mixed

Add Emissive Surfaces

- Baked GI
- Realtime GI
- None

Add Reflection Probes

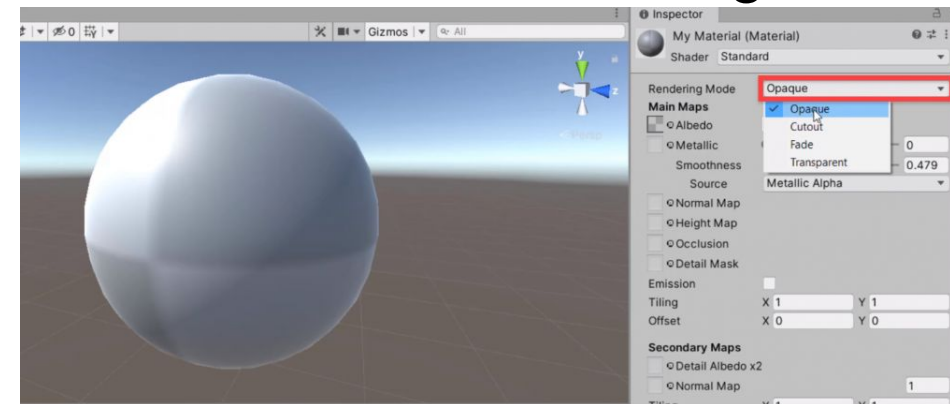
- Baked
- Realtime
- Custom

Add Light Probes

Add LPPVs (optional)

Materials

- **Materials** define how an **object** looks.
- The basic properties include:
 - **Albedo**: The **color/texture** of material.
 - **Metallic**: How metallic an object appears. High value will have more **reflection**.
 - **Smoothness**: How **smooth** a surface is.
 - **Normal maps**: Simulates **bumps and dents** without increasing polygon count.
 - **Emission**: Make material emit light.





Q&A Session



Attendance



Join us for the upcoming weekly sessions this Spring 2025!
(Jan 6 - Apr 4, 2025)



Monday: C5 1PM / C3 2PM
Tuesdays: C7 1PM / C11 3PM
Wednesdays: C11 10AM / C5 12:30 PM
C9 1:30 PM (Varies) C1 2:30PM
Thursdays: C6 1PM / C4 2PM / C8 3PM
Friday: C2 1PM / C9 2PM (Varies) / C10 3PM



RSVP [LU.MA/CODECRUNCH](https://lu.ma/codecrunch)



Your feedback helps us improve. Share your thoughts!
Go to our website: [CODECRUNCH.CS.FIU](https://codecrunch.cs.fiu.edu)